Refactorisation - FestiPlan

Présentation du projet

Contexte :

FestiPlan est une API de gestion d'un festival de musique fictif, inspiré de l'univers des grands festivals metal.

L'application permet aux organisateurs de gérer les informations du festival, ses scènes, les artistes invités et la programmation des concerts. Elle permet également de publier le planning et de déclarer des incidents survenus pendant les concerts.

Le projet est déjà fonctionnel, mais son architecture présente plusieurs défauts. Certaines responsabilités sont trop étroitement liées, certaines règles métier ne sont pas suffisamment protégées et certaines opérations peuvent conduire à des états incohérents.

Votre mission consiste à analyser, refactoriser et tester l'application, en vous appuyant sur l'architecture hexagonale et sur les concepts DDD lorsque ceux-ci sont pertinents.

Fonctionnalités existantes :

Fonctionnalité	Description
Festivals	Créer et consulter un festival, gérer son état
Artistes	Enregistrer des artistes et leurs besoins techniques
Scènes	Définir les scènes, leur capacité et leurs équipements
Concerts	Programmer, consulter, déplacer ou annuler des concerts
Planning	Consulter les concerts d'un festival dans l'ordre chronologique
Publication	Publier le planning destiné au public
Incidents	Enregistrer un incident associé à un concert
Notifications	Simuler l'envoi d'une notification lors de certaines opérations

Les artistes et les données fournies dans le projet sont fictifs.
Le jeu de données s'inspire de l'ambiance des festivals metal, sans représenter une programmation officielle.

Vocabulaire métier :

- **Festival** : événement global, caractérisé notamment par ses dates
- **Scène** : espace sur lequel se déroulent les concerts. Une scène appartient à un festival
- **Artiste** : groupe ou artiste pouvant être programmé
- **Concert** : représentation d'un artiste sur une scène pendant un créneau donné
- **Créneau** : intervalle défini par une heure de début et une heure de fin
- **Planning** : ensemble des concerts programmés pour un festival
- **Publication** : passage du planning préparatoire à un planning officiellement annoncé
- **Incident** : événement à signaler et à associer à un concert existant

Conception technique de départ

Le projet repose sur une API Python utilisant les technologies suivantes :

- FastAPI pour exposer les endpoints HTTP
- Pydantic pour valider les données reçues par l'API
- SQLAlchemy pour la persistance des données
- SQLite comme base de données locale par défaut
- pytest pour exécuter les tests

L'application est un projet monolithique : ses fonctionnalités sont réunies dans un même projet et exécutées dans une seule application. Ce n'est pas un problème en soi. Le problème est plutôt de savoir si les responsabilités sont correctement séparées à l'intérieur de ce projet.

Dans une architecture saine, les règles métier ne devraient pas dépendre directement de FastAPI ou de SQLAlchemy. Les détails HTTP et base de données doivent pouvoir évoluer sans obliger à réécrire le cœur métier.

Lancer et explorer l'application existante

Installation

Depuis un terminal ouvert à la racine du projet :

```
python -m venv .venv
```

Activez ensuite l'environnement virtuel.

Sous Windows PowerShell :

```
.venv\Scripts\Activate.ps1
```

Sous macOS ou Linux :

```
source .venv/bin/activate
```

Installez les dépendances :

```
pip install -r requirements.txt
```

Initialisez la base de données et les données de démonstration :

```
python -m app.seed
```

Lancez enfin l'API :

```
uvicorn app.main:app --reload
```

La documentation interactive de l'API est accessible à l'adresse suivante :

http://127.0.0.1:8000/docs

Pour exécuter les tests existants :

```
pytest
```

Attention : la commande d'initialisation des données peut réinitialiser les données de la base configurée. Utilisez-la uniquement dans votre environnement de développement.

Première étape : explorer et diagnostiquer

Avant de modifier le code, vous devez :

1. Parcourir les endpoints dans la documentation Swagger
2. Identifier les fichiers qui gèrent les requêtes HTTP
3. Repérer les endroits où sont exécutées les règles métier
4. Identifier le code responsable de la lecture et de l'écriture en base
5. Exécuter les tests existants
6. Reproduire les défauts connus et noter les étapes nécessaires pour les déclencher

À cette étape, vous devez comprendre comment l'application fonctionne réellement, et non comment vous pensez qu'elle devrait fonctionner.

Les problèmes à corriger

Trois défauts servent de points de départ à votre analyse.

- Défaut 1 — Chevauchement de concerts

Le déplacement d'un concert peut créer un chevauchement avec un autre concert programmé sur la même scène. Il faut également vérifier qu'un même artiste ne joue pas simultanément sur plusieurs scènes.

Objectif : centraliser et tester les règles de disponibilité du planning.

- Défaut 2 — Échec de notification après modification

Une modification peut être enregistrée en base, puis l'envoi d'une notification échouer. L'API peut alors renvoyer une erreur HTTP 500 alors que la modification a bien été réalisée.

Objectif : distinguer la réussite de l'opération métier de l'échec d'un effet secondaire.

- Défaut 3 — Modification d'un planning publié

Un concert peut être déplacé alors que le planning du festival est déjà publié, sans procédure métier particulière.

Objectif : gérer explicitement les états du planning et les transitions autorisées.

Ces trois défauts doivent être reproduits, documentés, puis couverts par des tests de non-régression. Il ne suffit pas de modifier le code en supposant qu'ils sont corrigés.

Les règles métier à respecter

Les règles suivantes constituent les exigences fonctionnelles de référence du projet. Elles doivent être prises en compte dans votre modèle métier et protégées par des tests.

Réf.	Règle métier
R1	L'heure de fin d'un concert doit être strictement postérieure à son heure de début
R2	Deux concerts ne peuvent pas se chevaucher sur une même scène
R3	Un artiste ne peut pas jouer dans deux concerts qui se chevauchent, même sur des scènes différentes
R4	Un concert doit référencer un festival, un artiste et une scène existants. La scène doit appartenir au festival du concert
R5	Les équipements requis par un artiste doivent être disponibles sur la scène choisie
R6	Un concert doit se dérouler entièrement dans la période du festival
R7	Un planning publié ne doit pas pouvoir être modifié par une opération ordinaire. Une procédure métier explicite est nécessaire
R8	Un planning ne peut être publié que si ses concerts respectent les règles de programmation
R9	La capacité d'une scène doit être strictement positive. Les durées et quantités obligatoires ne doivent pas être nulles ou négatives
R10	Un incident doit être associé à un concert existant et posséder une description non vide

Important : ces règles ne doivent pas être dispersées arbitrairement entre les contrôleurs HTTP, les services et les repositories. Vous devez réfléchir à l'endroit où chaque règle doit être portée et à la manière dont elle est protégée.

Par exemple, une validation du format d'une requête HTTP peut relever de Pydantic. En revanche, la règle interdisant deux concerts simultanés sur une même scène est une règle métier : elle doit rester applicable même si le cas d'utilisation est exécuté sans passer par HTTP.

Consignes pour mener à bien la refactorisation

Etape 1 - Analyser l'architecture initiale

Produisez un diagnostic succinct de l'application actuelle.

Vous devez identifier :

- les responsabilités de chaque grande partie du code
- les dépendances entre routes, services, modèles et accès aux données
- les endroits où se trouvent les règles métier
- les dépendances directes à FastAPI et SQLAlchemy
- les problèmes de testabilité et les risques d'incohérence

Présentez au moins un exemple concret de responsabilité mal placée ou de dépendance problématique.

Livrable intermédiaire : un diagnostic de l'architecture initiale, accompagné d'un schéma simple si cela facilite l'explication.

Étape 2 — Définir le modèle métier avec DDD

Vous devez réfléchir au domaine avant de décider comment répartir les fichiers.

Identifier les entités

Une entité possède une identité qui permet de la distinguer des autres objets et qui reste pertinente au cours de son cycle de vie.

Par exemple, un concert possède un identifiant. Deux concerts peuvent avoir les mêmes horaires, mais restent deux concerts distincts.

Identifier les objets-valeurs

Un objet-valeur représente une notion métier définie principalement par les valeurs qui la composent, plutôt que par une identité propre.

Exemples possibles :

- un créneau horaire ;
- une période de festival ;
- un ensemble de besoins techniques.

Un objet-valeur peut être immuable et vérifier sa validité à sa création.

Définir les invariants

Un invariant est une règle qui doit rester vraie pour qu'un objet ou un ensemble d'objets soit dans un état valide.

Exemple : un créneau dont l'heure de fin précède ou égale l'heure de début est invalide.

Vous devez identifier les invariants et décider où ils doivent être garantis : objet métier, agrégat, politique de domaine ou cas d'utilisation selon la règle concernée.

Définir les agrégats

Un agrégat regroupe des objets métier soumis à des règles de cohérence communes. Sa racine constitue le point d'entrée pour les modifications qui doivent préserver ces règles.

Vous devez proposer vos frontières d'agrégats et les justifier. Il n'est pas demandé de placer tous les objets dans un seul agrégat, ni de créer un agrégat par classe.

Réfléchir aux événements de domaine

Un événement de domaine représente un fait métier qui s'est produit.

Par exemple, *ConcertRescheduled* peut signaler qu'un concert a été déplacé. Une notification peut ensuite réagir à cet événement.

Vous devez distinguer :

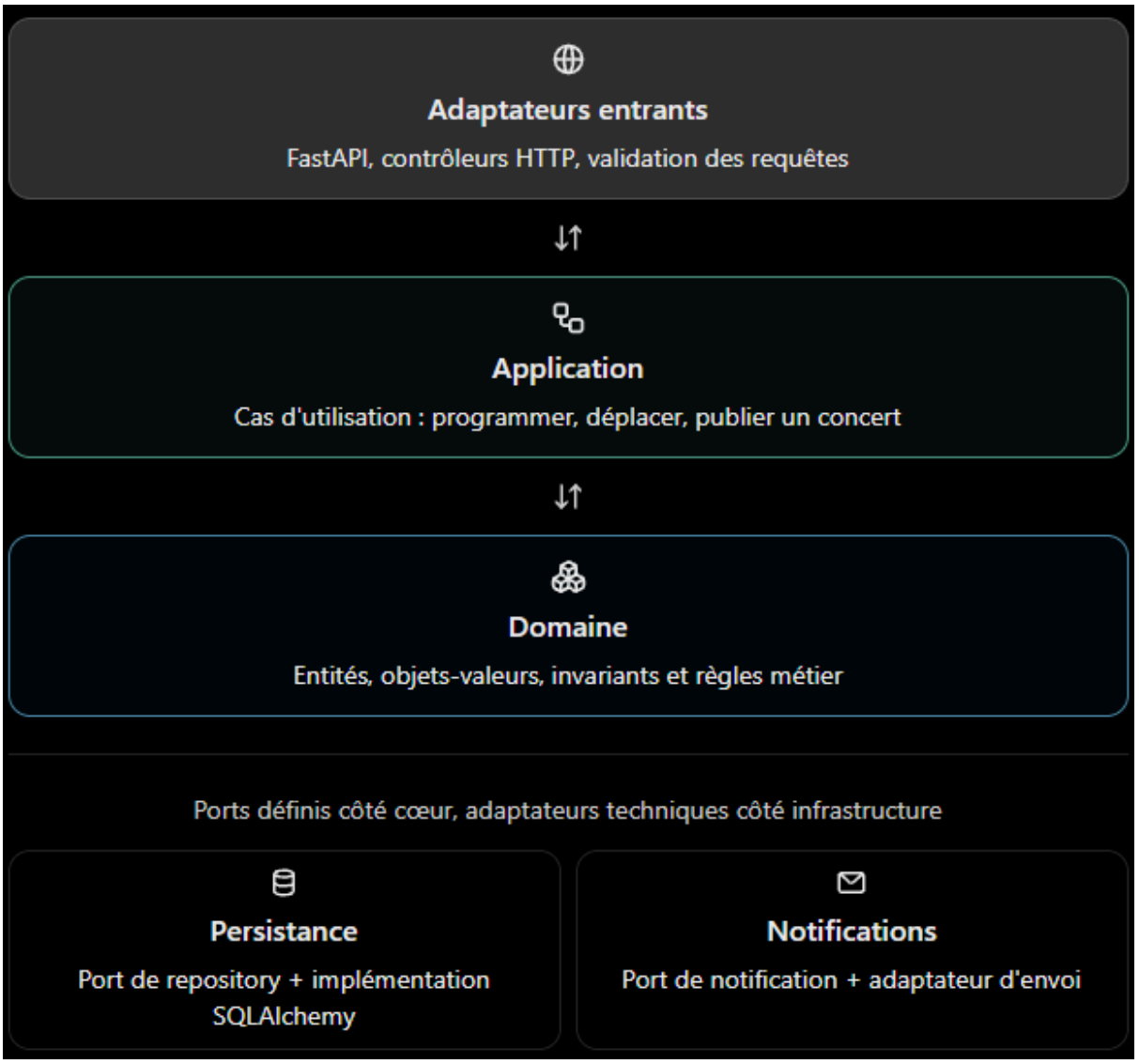
- la commande qui demande une action
- le cas d'utilisation qui orchestre cette action
- l'événement qui décrit le fait accompli
- le mécanisme technique qui réagit à cet événement

Livrable intermédiaire : une première modélisation du domaine, accompagnée d'explications sur les entités, objets-valeurs, invariants et agrégats retenus.

Étape 3 — Organiser le projet en architecture hexagonale

L'architecture hexagonale sépare le cœur de l'application des mécanismes techniques qui lui permettent de communiquer avec l'extérieur.

Une organisation possible est la suivante :



Ce schéma illustre une organisation possible, pas une arborescence obligatoire.

A. Le domaine

Il contient les concepts métier et les règles qui ne doivent pas dépendre du framework web ni de la base de données.

B. L'application

Elle contient les cas d'utilisation. Elle coordonne les objets métier, demande les données nécessaires via des ports et orchestre les opérations.

Exemples de cas d'utilisation :

- programmer un concert
- déplacer un concert
- publier un planning
- enregistrer un incident

C. Les ports

Un port définit un contrat dont le cœur applicatif a besoin ou un point d'entrée de l'application.

Par exemple, un port de repository peut définir les opérations nécessaires pour retrouver et enregistrer un concert. Le code applicatif dépend du contrat, et non de l'implémentation SQLAlchemy.

D. Les adaptateurs

Les adaptateurs connectent l'application aux technologies concrètes :

- un contrôleur FastAPI adapte une requête HTTP à un cas d'utilisation
- un repository SQLAlchemy adapte la persistance à un port
- un adaptateur de notification connecte le domaine applicatif à un système d'envoi

Attention : un contrôleur HTTP est un adaptateur entrant ; il n'est pas nécessairement le port lui-même. De même, le repository SQLAlchemy est une implémentation d'un port sortant, pas le port.

E. L'assemblage des dépendances

L'application doit disposer d'un endroit où les dépendances concrètes sont créées et injectées : par exemple un module de composition ou de démarrage.

Les cas d'utilisation ne doivent pas instancier eux-mêmes les repositories SQL ou les clients de notification. Ils doivent recevoir les dépendances dont ils ont besoin.

Étape 4 — Refactoriser sans casser le comportement

Procédez par petits changements vérifiables.

1. Commencez par une fonctionnalité précise, comme le déplacement d'un concert
2. Identifiez le comportement existant et les tests nécessaires
3. Isolez les règles métier concernées
4. Créez ou adaptez le cas d'utilisation
5. Définissez les ports nécessaires et leurs implémentations techniques
6. Adaptez le contrôleur HTTP pour appeler le cas d'utilisation
7. Exécutez les tests et vérifiez que l'API continue de fonctionner
8. Répétez la démarche pour les autres fonctionnalités

Évitez de déplacer tout le code d'un coup. Une refactorisation progressive permet de repérer plus facilement les régressions et de comprendre les bénéfices de chaque changement.

Étape 5 — Corriger les trois défauts initiaux

Défaut 1 : conflits de programmation

Le déplacement d'un concert doit vérifier les conflits avec les autres concerts concernés.

Les tests doivent au minimum vérifier :

- qu'un chevauchement sur une même scène est refusé
- qu'un artiste ne peut pas jouer simultanément sur deux scènes
- qu'un déplacement valide est accepté

Défaut 2 : notification en échec

La réussite de la modification métier doit être distinguée du succès de l'envoi de la notification.

Vous devez définir le comportement attendu si la modification est enregistrée, mais que la notification échoue. Une réponse HTTP 500 ne doit pas laisser penser que rien n'a été modifié si la modification a en réalité été persistée.

Vous pouvez proposer une solution proportionnée au projet. Une gestion asynchrone avancée n'est pas obligatoire, mais votre décision doit être expliquée et testée.

Défaut 3 : planning publié

La publication doit être représentée par un état métier cohérent. Les opérations autorisées doivent dépendre de cet état.

Vous devez définir le comportement d'une demande de déplacement après publication : refus, dépublication préalable ou procédure spécifique de modification, selon la règle métier que vous retenez.

Ce comportement doit être explicite, cohérent et couvert par des tests.

Étape 6 — Mettre en place une stratégie de tests

Votre suite de tests doit permettre de vérifier le domaine et les cas d'utilisation indépendamment des technologies utilisées.

Niveau de test	Ce qu'il faut vérifier
Tests unitaires du domaine	Créneaux invalides, chevauchements, invariants et transitions d'état
Tests des cas d'utilisation	Programmation, déplacement, publication, gestion des erreurs et interactions avec les ports
Tests des adaptateurs	Fonctionnement des routes HTTP et de la persistance
Tests de non-régression	Correction des trois défauts initiaux sans réintroduire les problèmes

Les tests du domaine ne devraient pas nécessiter de démarrer FastAPI ni de se connecter à SQLite.

Les tests applicatifs peuvent utiliser des doubles de test ou des repositories en mémoire pour vérifier les décisions métier et les appels aux dépendances.

Il n'est pas nécessaire de tester chaque ligne de code. Privilégiez les règles métier importantes, les erreurs possibles et les comportements qui doivent rester garantis.

Documentation technique attendue

Diagramme de composants

Le diagramme doit montrer les principales parties de votre architecture et leurs dépendances.

Il doit permettre de comprendre :

- où se trouvent le domaine et les cas d'utilisation
- quels ports sont définis
- quels adaptateurs les implémentent
- comment les requêtes HTTP atteignent le cœur applicatif
- comment la persistance et les notifications sont branchées

Le diagramme doit représenter votre architecture réelle, pas seulement une architecture théorique.

Diagramme de séquence

Choisissez un cas d'utilisation important. Le déplacement d'un concert est recommandé, car il implique la vérification des règles de programmation, la persistance et potentiellement une notification.

Le diagramme doit représenter les interactions, les vérifications métier, les éventuels refus et le résultat retourné.

Si vous utilisez des événements ou un mécanisme asynchrone, faites apparaître clairement à quel moment la modification est validée et comment l'échec d'une notification est traité.

Note d'architecture

Votre note doit expliquer :

- les principaux défauts identifiés dans l'architecture initiale
- les responsabilités retenues pour chaque couche
- les ports et adaptateurs créés et leur utilité
- les entités, objets-valeurs, invariants et agrégats retenus
- la manière dont les trois défauts initiaux ont été corrigés
- la stratégie de tests
- les limites restantes et les compromis réalisés

L'objectif est de démontrer que vous comprenez les conséquences de vos choix, pas simplement de décrire votre arborescence.

Livrables finaux

À la fin du projet, vous devez remettre :

- Le code refactorisé et exécutable
- Les tests unitaires, applicatifs et d'intégration pertinents
- Les tests de non-régression pour les trois défauts initiaux
- Un diagramme UML de composants
- Un diagramme UML de séquence
- Une note d'architecture justifiant les choix effectués
- Un README mis à jour, avec les instructions d'installation, d'initialisation, de lancement et de test

Conseils pour réussir

- **Comprenez avant de modifier :** Un fichier mal nommé n'est pas nécessairement le problème principal. Cherchez surtout les responsabilités et dépendances inadaptées
- **Testez avant et après :** Un test qui échoue avant la correction et réussit ensuite constitue une preuve utile
- **Ne surconcevez pas :** Une abstraction doit répondre à un besoin réel, pas simplement augmenter le nombre de classes
- **Protégez les règles au bon endroit :** Une validation HTTP seule ne garantit pas la cohérence du domaine
- **Justifiez DDD :** Les concepts DDD doivent rendre le modèle plus clair et protéger des règles métier, pas être ajoutés uniquement pour respecter une liste
- **Gardez le projet exécutable :** Chaque étape de refactorisation doit pouvoir être vérifiée

Le résultat attendu est une application dont les règles métier sont plus fiables, dont les dépendances sont mieux maîtrisées et dont les tests permettent de faire évoluer le code avec davantage de confiance.

Il n'existe pas une seule architecture correcte : la cohérence des choix, leur justification et la capacité à démontrer leur efficacité sont essentielles.